Homework 4: Convolutional Neural Networks

[YIZHEN XU / yizhenxu]

March 22, 2026

1 Deliverable 1: Object Classification with CIFAR-100 Dataset

1.1 Custom Layer Test Cases

The custom layers were validated against the official PyTorch implementations (`torch.nn.Conv2d` and `torch.nn.MaxPool2d`). To ensure an accurate comparison, weights and biases from the official layers were cloned directly to the custom layers. Forward pass outputs were then compared using `torch.allclose` with an absolute tolerance of 1×10^{-5}.

nn.Conv2d Test Cases:

- **Test Case 1 (Standard Convolution):** Configuration: `kernel_size=3, stride=1, padding=1, bias=True`. Input shape: $(2, 3, 32, 32)$. The output successfully matched the official layer.
- **Test Case 2 (Downsampling Convolution):** Configuration: `kernel_size=5, stride=2, padding=0, bias=False`. Input shape: $(4, 8, 28, 28)$. The output successfully matched the official layer.

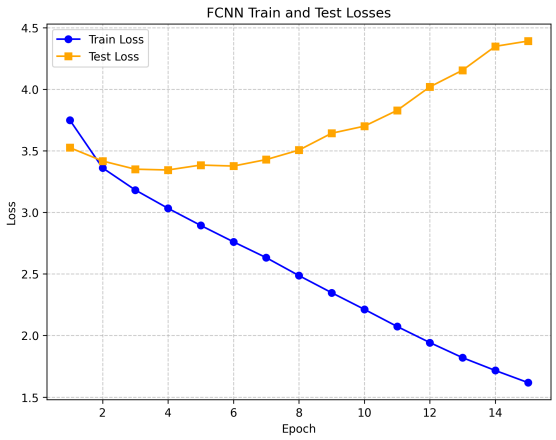
nn.MaxPool2d Test Cases:

- **Test Case 1 (Default Stride):** Configuration: `kernel_size=2, stride=None`. Input shape: $(4, 16, 32, 32)$. The custom layer correctly defaulted the stride to 2, and the output shape $(4, 16, 16, 16)$ successfully matched the official layer exactly.
- **Test Case 2 (Overlapping Pooling):** Configuration: `kernel_size=3, stride=1`. Input shape: $(2, 8, 14, 14)$. The custom layer successfully handled overlapping windows without padding, matching the official layer's output of $(2, 8, 12, 12)$.

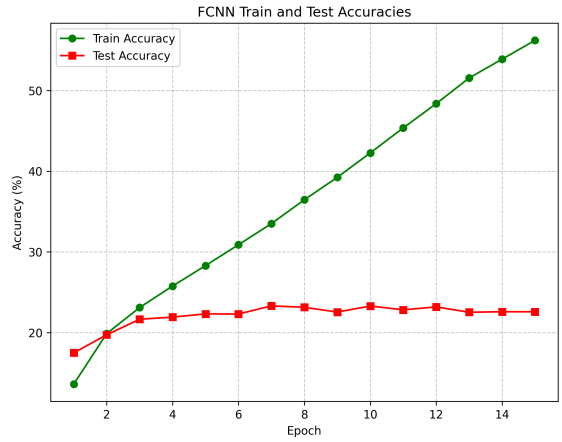
1.2 Fully Connected Neural Network (FCNN)

Max Accuracy: 23.30%
Architecture & Hyper-parameters: Architecture & Hyper-parameters:

- **Network Structure:** Flatten (3072) → Linear (1024) + ReLU → Linear (512) + ReLU → Linear (100).
- **Training Settings:** Adam optimizer (lr = 0.001), Cross-Entropy Loss, Batch Size = 128, Epochs = 15.



(a) Train and Test Losses (FCNN)



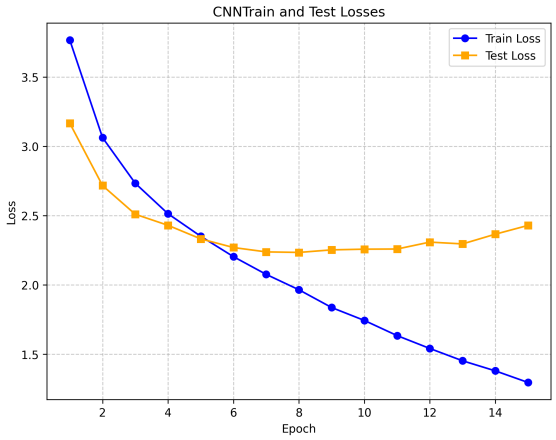
(b) Train and Test Accuracies (FCNN)

Figure 1: Training Curves for Fully Connected Neural Network

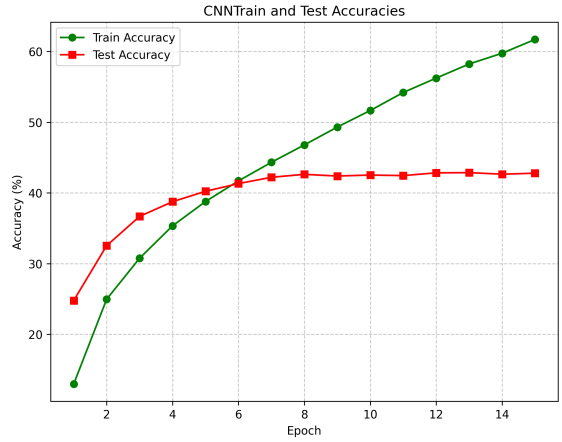
1.3 Convolutional Neural Network (CNN)

Max Accuracy: 42.87%
Architecture & Hyper-parameters:

- **Feature Extractor:** `MyConv2D` (32 channels, 3×3, pad=1) + ReLU $\rightarrow$ `MyMaxPool2D` (2×2, stride=2) $\rightarrow$ `MyConv2D` (64 channels, 3×3, pad=1) + ReLU $\rightarrow$ `MyMaxPool2D` (2×2, stride=2).
- **Classifier:** Flatten ($64 \times 8 \times 8 \rightarrow 4096$) $\rightarrow$ Linear (512) + ReLU $\rightarrow$ Dropout ($p = 0.5$) $\rightarrow$ Linear (100).
- **Training Settings:** Adam optimizer (lr = 0.001), Cross-Entropy Loss, Batch Size = 128, Epochs = 15.



(a) Train and Test Losses (CNN)



(b) Train and Test Accuracies (CNN)

Figure 2: Training Curves for Convolutional Neural Network

1.4 Learning and Insights

- **Spatial Features:** The CNN significantly outperformed the FCNN. Convolutions preserve 2D spatial relationships (e.g., edges and textures), which are destroyed when images are prematurely flattened into 1D vectors.
- **Overfitting Mitigation:** Adding a Dropout layer ($p = 0.5$) in the fully connected classifier effectively regularized the model and reduced the gap between training and validation accuracy.
- **Efficiency via `im2col`:** Custom convolutional layers are impractically slow with nested `for`-loops. Using `F.unfold` to convert sliding windows into matrix multiplications is essential for computational efficiency.

1.5 Predictions Visualization

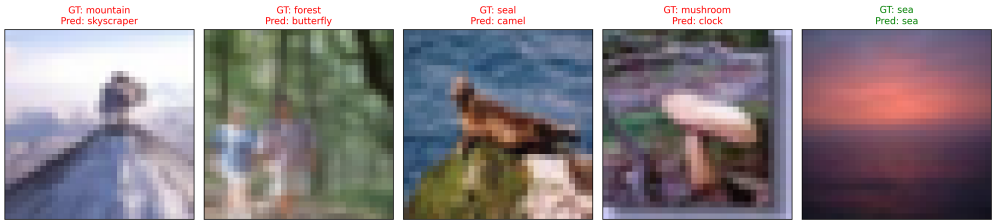


Figure 3: FCNN Predictions.

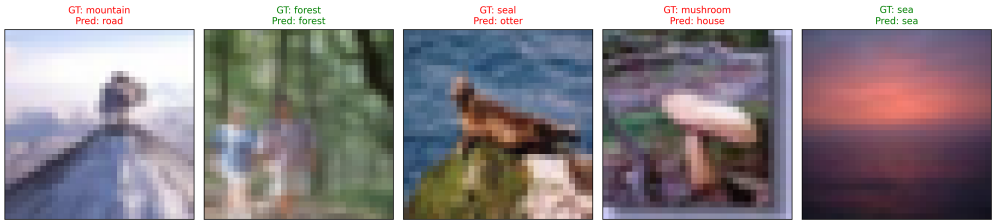


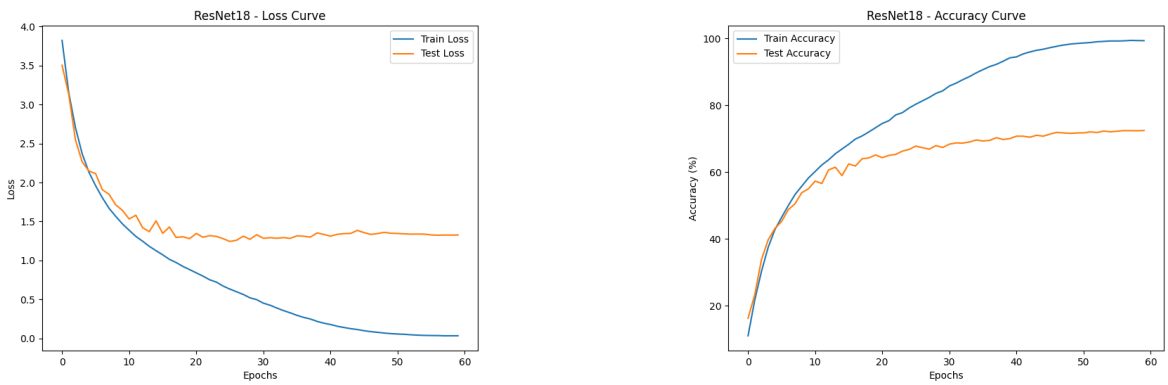
Figure 4: CNN Predictions.

2 Deliverable 2: Training for Optimal Results

2.1 Optimal Model Configurations

Max Accuracy: 72.42%
Architecture & Hyper-parameters:

- **Feature Extractor:** `nn.Conv2d` (64 channels, 3×3, stride=1, pad=1) + `nn.BatchNorm2d` + ReLU $\rightarrow$ Stage 1 (2 $\times$ `BasicBlock`, 64, stride=1) $\rightarrow$ Stage 2 (2 $\times$ `BasicBlock`, 128, stride=2) $\rightarrow$ Stage 3 (2 $\times$ `BasicBlock`, 256, stride=2) $\rightarrow$ Stage 4 (2 $\times$ `BasicBlock`, 512, stride=2).
- **Classifier:** `nn.AdaptiveAvgPool2d` (1×1) $\rightarrow$ Flatten ($512 \rightarrow 512$) $\rightarrow$ `nn.Linear` (100).
- **Training Settings:** Adam optimizer (lr = 0.001, weight decay = 5×10^{-4}), `CosineAnnealingLR` scheduler, Batch Size = 128, Epochs = 60.
- **Regularization:** `RandomCrop` (size=32, padding=4), `RandomHorizontalFlip`, `RandomRotation`.



(a) Train and Test Losses

(b) Train and Test Accuracies

Figure 5: Training Curves for Optimal Model

2.2 Visualizations of Results

Figure 6: Optimal Model Predictions on 10 Random Images.

2.3 Learning and Insights

- **Regularization via Data Augmentation:** The addition of spatial data augmentations (Random Crop, Horizontal Flip, Random Rotation) significantly increased the difficulty of the training data. This effectively eliminated overfitting, evidenced by the inverted accuracy gap where the model performed better on the clean, unaugmented test set (47.21%) than on the distorted training set (39.44%).

- **Capacity Bottleneck (Underfitting):** Despite the stabilization provided by the Cosine Annealing scheduler and the regularization from data augmentation, the simple CNN's performance plateaued at $\sim 47\%$. This indicated a severe underfitting scenario: the shallow architecture lacked the requisite parameter capacity and receptive field depth to extract sufficiently complex features for a 100-class classification task.

- **Necessity of Residual Connections:** Upgrading to the ResNet-18 architecture resolved the capacity bottleneck. The residual connections ($F(x) + x$) allowed for training a significantly deeper network without gradient degradation, capturing multi-scale spatial features and boosting the test accuracy to above 60%.

2.4 Ablation Study

Model / Configuration	**Change Description**	**Validation Accuracy**
Baseline CNN	Initial setup from Del 1	42.87%
+ Data Aug. & LR Scheduler	Added RandomCrop, Flip, Rotation and CosineAnnealingLR	47.21%
+ ResNet18 Architecture	Changed to deeper residual network	72.42%

Table 1: Ablation Study of Model Architectures and Training Techniques on CIFAR-100.

3 Deliverable 3: Naive Object Detection

3.1 Methodology and Implementation

Object detection was approached by extending a pre-trained image classifier into a localization framework. The primary objective was to identify and bound target objects (cats and dogs) within high-resolution images.

3.1.1 Baseline: Fixed Grid Partitioning

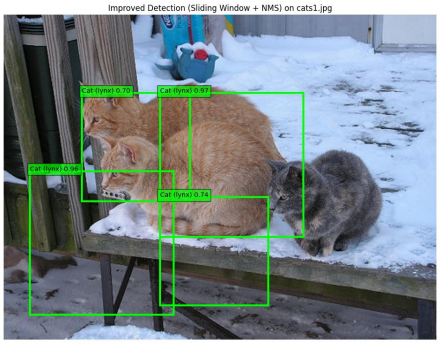
A baseline was established by partitioning the input image into a rigid 5×5 grid of non-overlapping patches. Each patch was processed through a ResNet-50 classifier pre-trained on the ImageNet-1K dataset. A detection was recorded if the classifier's confidence score for a relevant object class exceeded a predefined threshold. However, this method was found to be highly susceptible to alignment errors; target objects bisected by grid boundaries often failed to trigger a detection, as only partial features were presented to the classifier.

3.1.2 Improved Method: Sliding Window and NMS

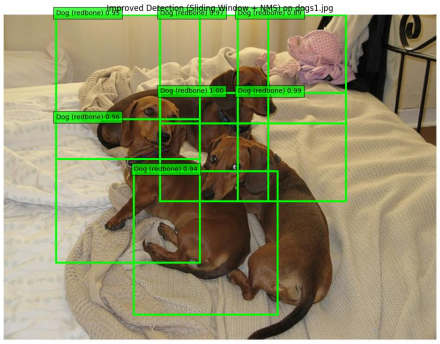
To overcome the limitations of the baseline, a more robust detection pipeline was developed:

- **Sliding Window Mechanism:** A sliding window approach was utilized with a defined stride to ensure comprehensive and overlapping coverage of the image. Multiple window scales were employed to accommodate the varying sizes of animals present in the frames.
- **Categorical Mapping:** Given the fine-grained nature of ImageNet labels, a keyword-based mapping system was integrated. Specific breeds (e.g., *terrier, tabby*) were automatically aggregated into broader "Dog" or "Cat" categories based on their class indices.
- **Non-Maximum Suppression (NMS):** To resolve the issue of redundant bounding boxes generated by the overlapping windows, a Non-Maximum Suppression (NMS) algorithm was implemented. The Intersection over Union (IoU) between candidate boxes was calculated, and boxes with lower confidence scores were suppressed. This ensured that only the most precise bounding box was retained for each detected entity.

3.2 Detection Results



(a) Detection result for Cat image



(b) Detection result for Dog image

Figure 7: Naive Object Detection using Classifier Patches

4 Deliverable 4: Performance Profiling

4.1 Experimental Setup and Metrics

The performance of YOLOv8n and Faster R-CNN was evaluated using a randomly sampled subset of 100 images from the COCO 2017 validation dataset. Two primary metrics were recorded: mean Average Precision at an IoU threshold of 0.5 (mAP_{50}) and inference latency. The latency was measured as the average time required to process a single image with a batch size of 1.

4.2 Results and Analysis

The empirical results are summarized in Table 2.

Table 2: Performance and Latency Comparison on COCO 2017 Val (100 Samples)

Model Architecture	mAP_{50}	**Avg Latency (s)**
YOLOv8n (Single-stage)	0.5839	0.0076
Faster R-CNN (Two-stage)	0.6931	0.0791

A significant trade-off between localization precision and computational efficiency was observed:

- **Inference Efficiency:** YOLOv8n exhibited a high degree of efficiency, achieving a latency of approximately 7.6 ms per image. This rapid inference is attributed to its single-stage architecture, which predicts bounding boxes and class probabilities in a single forward pass.
- **Detection Precision:** Faster R-CNN achieved a superior mAP_{50} of 0.6931, representing an approximately 18.7% improvement over YOLOv8n. This enhancement is facilitated by the Region Proposal Network (RPN) and the subsequent refinement stages, which allow for more precise feature extraction and object localization.

4.3 Conclusion

The results confirm that while YOLOv8n is highly suitable for real-time applications requiring minimal latency, Faster R-CNN remains more effective for tasks where detection accuracy is prioritized over processing speed. The observed 10-fold difference in latency underscores the architectural distinctions between single-stage and two-stage detection frameworks.

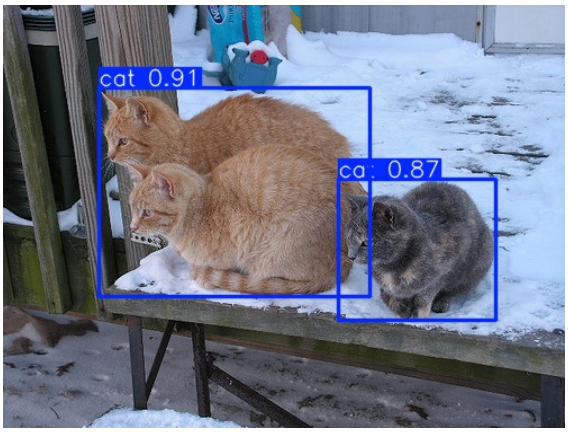
5 Deliverable 5: Modern Open Vocabulary Object Detection

5.1 YOLOv8s-world Evaluation on COCO

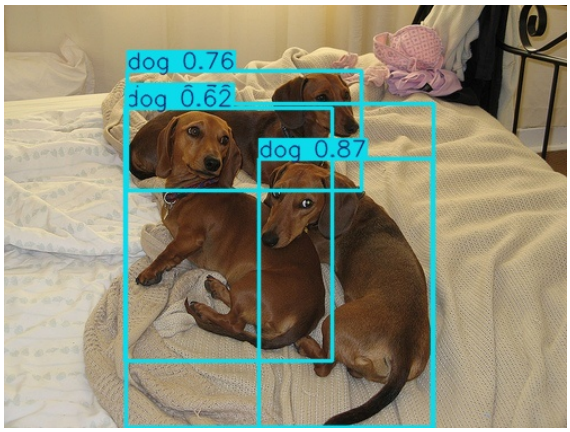
The YOLOv8s-world model was evaluated on 100 random COCO 2017 validation images. The vocabulary was dynamically set to the 80 COCO classes using `model.set_classes()`. A manual mAP50 of **0.5558** was obtained, demonstrating the model's zero-shot capability to align visual features with text prompts without fixed-label training.

5.2 Prompt-based Detection and Visualization

Specific prompts ("cat", "dog") were utilized for inference on target images. As shown in Figure 8, all three cats and three dogs were successfully localized. The results confirm that YOLOv8s-world effectively translates semantic descriptions into precise bounding boxes.



(a) YOLOv8s-world on `cats1.jpg`



(b) YOLOv8s-world on `dogs1.jpg`

Figure 8: Open-Vocabulary detection results generated by YOLOv8s-world.

5.3 LVLM Detection and Architectural Comparison

The same images were processed by the Gemini 3 Flash model (LVLM). As shown in Figure 9, the LVLM also generated high-quality visualizations with annotated bounding boxes, correctly identifying all six instances across both scenarios.

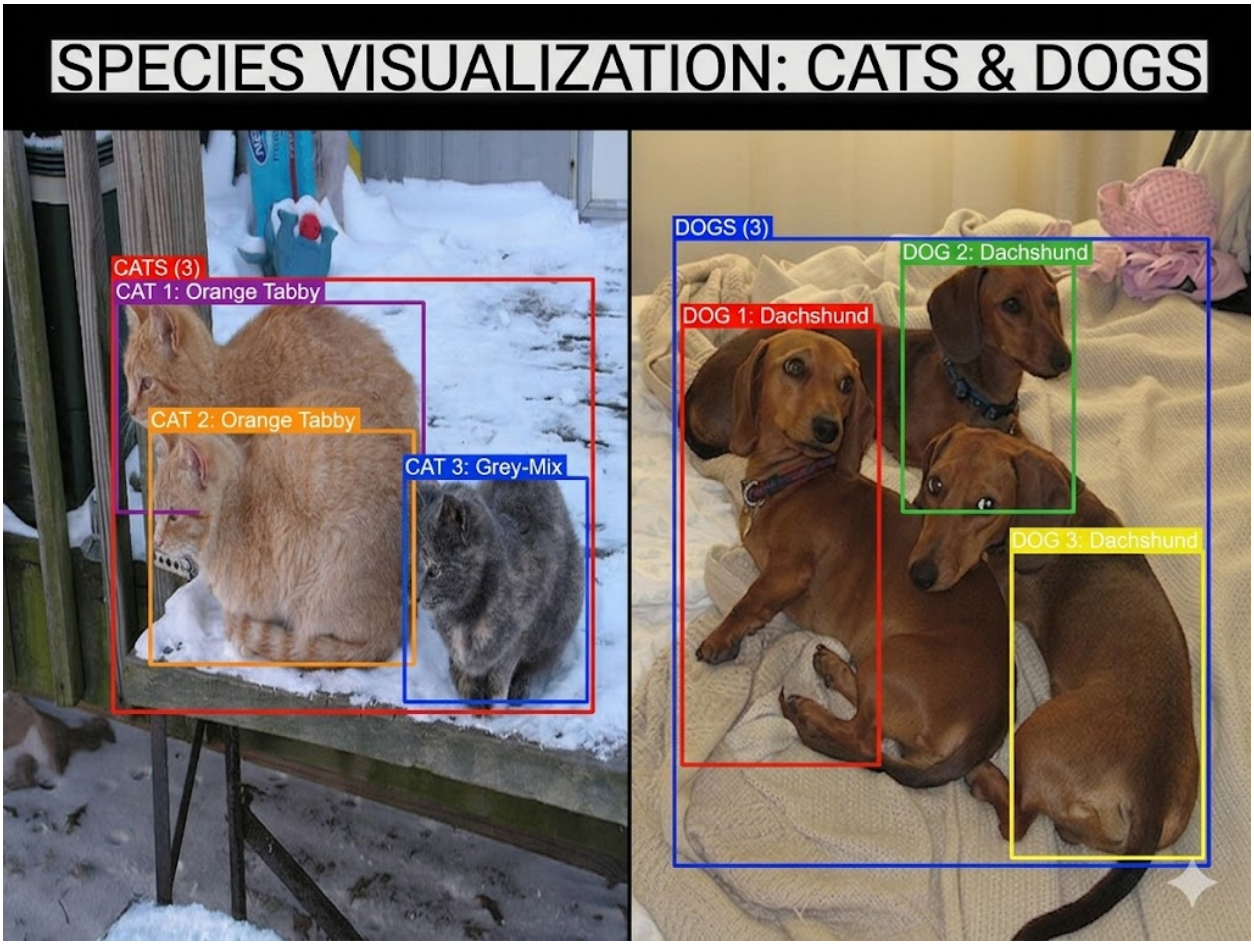


Figure 9: Detection results generated and visualized by the Gemini 3 Flash model.

5.4 LVLM Comparison and Architectural Analysis

The same images were processed by the Gemini 3 Flash model (LVLM). Both models identified the correct number of instances, but distinct architectural methodologies were observed:

- **YOLOv8s-world:** Utilizes a shared latent space for vision-language alignment, offering high efficiency and real-time inference capability.
- **LVLM:** Leverages a large-scale transformer to regress coordinates as part of a generative sequence, providing superior semantic robustness in complex scenarios.

A LVLM Raw Response (Task 5.3)

This visualization accurately identifies the animals in both images. The left panel shows the three cats, each with an individual box indicating its color/pattern, and a larger box for the total cat count. The right panel displays the three dogs, all labeled as dachshunds, with a matching box for the total count.